



Programmieren

Falk Morgenstern
Leon Johann Brettin

Foliensatz basierend auf Unterlagen von Prof. Huhn, Prof. Meyer und weiteren.
Nur zur Verwendung in der Vorlesung „Programmieren“ an der Ostfalia.
Keine Weitergabe.



Programmieren

11. Threads



Programmieren

11. Threads

11.1 Einführung



Nebenläufigkeit und Threads

- **Nebenläufigkeit** ist die Eigenschaft eines Programms, **mehrere Anweisungen gleichzeitig** ausführen zu können.
 - Nebenläufig = scheinbar oder tatsächlich parallel.
- In Java wird Nebenläufigkeit über **Threads** erzielt.
- Ein Thread ist ein Programmstück (eine Folge von Instruktionen), das nebenläufig zu anderen Instruktionsfolgen ablaufen kann.
- Eine Gruppe von Threads wird nebenläufig ausgeführt, wenn die einzelnen Threads in keiner kausalen Abhängigkeit stehen.
 - D.h. keine Notwendigkeit zur Abstimmung bei der Ausführung, z.B. hinsichtlich Kommunikation, Verwendung gemeinsamer Daten, verteilt ausgeführter Berechnungen

Nebenläufig, aber nicht
(nur „scheinbar“) parallel



Nebenläufig und parallel





Motivation für Nebenläufigkeit

- Auch innerhalb eines Programms gibt es Gründe für eine nebenläufige Ausführung:
 - die GUI soll auf Ereignisse des Benutzers reagieren, auch wenn im Hintergrund aufwendige Berechnungen laufen,
 - es bestehen verschiedene Netzwerkverbindungen, auf die gleichzeitig reagiert werden soll,
 - es stehen mehrere Cores zur Verfügung, die genutzt werden können.
- Threads beziehen sich auf die virtuelle Komponente, die die auf mehreren Kernen / Prozessoren parallel auszuführenden Aufgaben managt.
- Vorteile: Beschleunigte Ausführung, erhöhter Durchsatz.



Threads = Leichtgewichtige Prozesse

- **Thread:** Programmstück (eine Folge von Instruktionen), das **nebenläufig** zu anderen Instruktionsfolgen ablaufen kann; **leichtgewichtiger Prozess**
 - Ein Prozess kann mehrere Threads haben, sie teilen Adressraum, File Handles...
 - Jeder Thread besitzt seinen eigenen Stack, auf dem er lokale Variablen und Methodenaufrufe verwaltet.
 - Alle Threads eines Prozesses (Java-Programms) laufen in einer gemeinsamen Laufzeitumgebung (JVM) und teilen sich einen gemeinsamen Speicherbereich für globale Variablen und Objekte des Programms.
- **Konsequenzen:**
 - Keine Interprozesskommunikation (Betriebssysteme und Rechnernetze) notwendig.
 - Mögliche **Probleme beim nebenläufigen Zugriff** auf Daten müssen die Threads untereinander regeln.



Sequentielle Abarbeitung: Druckaufträge

- Beim Start eines Java-Programms wird ein Thread gestartet.
 - Dieser führt die main-Methode aus.
 - Innerhalb eines Threads werden Methodenaufrufe sequentiell ausgeführt.

```
public class MehrmalsP {  
    public static void main(String[] args) {  
        ABCPrinter p1 = new ABCPrinter(),  
                    p2 = new ABCPrinter();  
        p1.start();  
        p2.start();  
    }  
}
```





Beispiel: Druckaufträge



```
public class ABCPrinter {  
    public void run() {  
        for (char b = 'A'; b <= 'Z'; b++) {  
            // Gib den Buchstaben aus  
            System.out.print(b);  
            // Verbringe 100 Millisekunden mit "Pause"  
            MachMal.pause(100);  
        }  
        System.out.println();  
    }  
    public void start() {  
        run();  
    }  
}
```

- Ausgabe:

ABCDEFGHIJKLMNOPQRSTUVWXYZ

ABCDEFGHIJKLMNOPQRSTUVWXYZ



Aufteilung auf mehrere Threads: Fragestellungen

- Wie können Threads **gestartet** und **beendet** werden?
- Welche **Zustände** können Threads einnehmen?
- Welche Probleme können durch den **gleichzeitigen Zugriff mehrerer Threads auf dieselben Daten** auftreten?
- Wie können solche Probleme **vermieden** werden?
- Wie können Threads **koordiniert** an einer gemeinsamen Aufgabe arbeiten?



Threads in Java

- Threads werden durch Objekte der Klasse `java.lang.Thread` repräsentiert.
- Die Klasse `Thread` implementiert das Interface `java.lang.Runnable`, das die Methode `void run()` vereinbart. Diese Methode bestimmt den als Thread auszuführenden Code.
- Erzeugung von Threads:
 - Eine Klasse schreiben, die die Anweisungen des Threads definiert, und die
 1. von `Thread` abgeleitet ist, oder
 2. die direkt das `Interface Runnable` implementiert. Mit einem Objekt dieser Klasse als Konstruktor-Argument wird ein Thread-Objekt erzeugt. Diese Möglichkeit muss immer dann genutzt werden, wenn die Klasse selbst bereits Subklasse einer anderen Klasse ist.
- In beiden Fällen muss die Methode `run()` implementiert werden, die den Programmcode des Threads enthält. Die Thread-Methode `start()` sorgt für die Ausführung des Threads, indem sie selbst die Methode `run()` aufruft.



Erstes Beispiel für Threads: Variante 1

- Der ABCPrinter (nunmehr ABCThread) wird als Unterklasse von `java.lang.Thread` implementiert

```
public class ABCThread extends Thread {  
    @Override  
    public void run() {  
        for (char b = 'A'; b <= 'Z'; b++) {  
            // Gib den Buchstaben aus  
            System.out.print(b);  
            // Verbringe 100 Millisekunden mit "Pause"  
            MachMal.pause(100);  
        }  
        System.out.println();  
    }  
}
```





Erstes Beispiel für Threads: Variante 1

- Nun werden mehrere Threads gestartet

```
public class ManyThreads {  
    public static void main(String[] args) {  
        ABCThread t1 = new ABCThread(),  
            t2 = new ABCThread();  
        t1.start();  
        t2.start();  
    }  
}
```

- Ausgabe:

AABBCCDDEEFFGGHHIIJJKKLLMMNNOOPPQQRRSSTTUUVVWWXXYYZZ



Erstes Beispiel für Threads: Variante 2

- Anstelle einer Unterklasse von Thread kann man auch die Schnittstelle Runnable direkt implementieren

```
public class ABCRunnable implements Runnable {  
    @Override  
    public void run() {  
        for (char b = 'A'; b <= 'Z'; b++) {  
            // Gib den Buchstaben aus  
            System.out.print(b);  
            // Verbringe 100 Millisekunden mit "Pause"  
            MachMal.pause(100);  
        }  
        System.out.println();  
    }  
}
```





Erstes Beispiel für Threads: Variante 2

- ... und dann in `main` `Runnable`s erzeugen, die mit Hilfe von `Thread`-Objekten gesteuert werden können

```
public class ManyRunnables {  
    public static void main(String[] args) {  
        Thread t1 = new Thread(new ABCRunnable()),  
            t2 = new Thread(new ABCRunnable());  
        t1.start();  
        t2.start();  
    }  
}
```





Threads in Java: Erläuterungen

- Die Instruktionsfolge, die parallel ausgeführt werden soll, muss in der Methode `run()` enthalten sein, oder aber von `run()` (indirekt) aufgerufen werden.
- Mit einem Aufruf der Methode `start()` wird ein nebenläufiger Ausführungsfaden (engl. Thread) in den Zustand „runnable“ versetzt, d.h. er kann vom Scheduler der JVM dem nächsten frei werdenden Prozessor zugeteilt werden.
- Ein direkter Aufruf von `run()` entspricht einem sequentiellen Methodenaufruf, es wird also kein neuer Thread gestartet.
- Thread bietet eine Implementierung von `start()`
- die Standardimplementierung von `run()` in der Klasse Thread tut nichts (leerer Anweisungsblock).



Programmieren

11. Threads

11.2 Die Klasse Thread in Java



Die Klasse Thread

- Ein Thread ist in Java ein Objekt der Klasse `java.lang.Thread`.
- Konstruktor (u.a.):
 - `Thread(Runnable t)`: Durch den Aufruf von `start()` wird die `run()`-Methode von `t` nebenläufig zur Ausführung gebracht
- Instanz-Methoden (u.a.):
 - `public void start()`: Starten der Ausführung des Thread-Objekts
 - `public void run()`: beinhaltet den Teil, der nebenläufig auszuführen ist.
 - `public final boolean isAlive()`: liefert `true`, wenn der Thread gestartet, aber noch nicht beendet wurde.
 - `public void interrupt()`: setzt den interrupted status auf `true`.
 - `public boolean isInterrupted()`: gibt den interrupted status des Threads zurück, an dem die (Instanz-) Methode aufgerufen wurde. Der interrupted status dieses Threads wird durch diese Methode nicht beeinflusst. Wird i.a. von einem Thread benutzt, um den interrupted status eines anderen Threads abzufragen.
 - `public void join() throws InterruptedException`: wartet auf das Ende des Threads; wird verwendet, wenn ein Thread auf die Beendigung eines anderen Threads warten soll.



Die Klasse Thread

- **Klassenmethoden:**
 - `public static void sleep(long millis)` throws `InterruptedException`: verzögert die Ausführung des Threads um `millis` Millisekunden
 - `public static Thread currentThread()`: liefert eine Referenz auf den Thread, der gerade ausgeführt wird.
 - `public static void yield()`: lässt den Thread, der gerade ausgeführt wird, kurz pausieren, damit andere Threads zum Zuge kommen können.
 - `public static boolean interrupted()`: gibt den interrupted status des Threads zurück, der gerade ausgeführt wird (Klassenmethode!), und setzt diesen unmittelbar auf `false`



Verwendung der Klassenmethode sleep

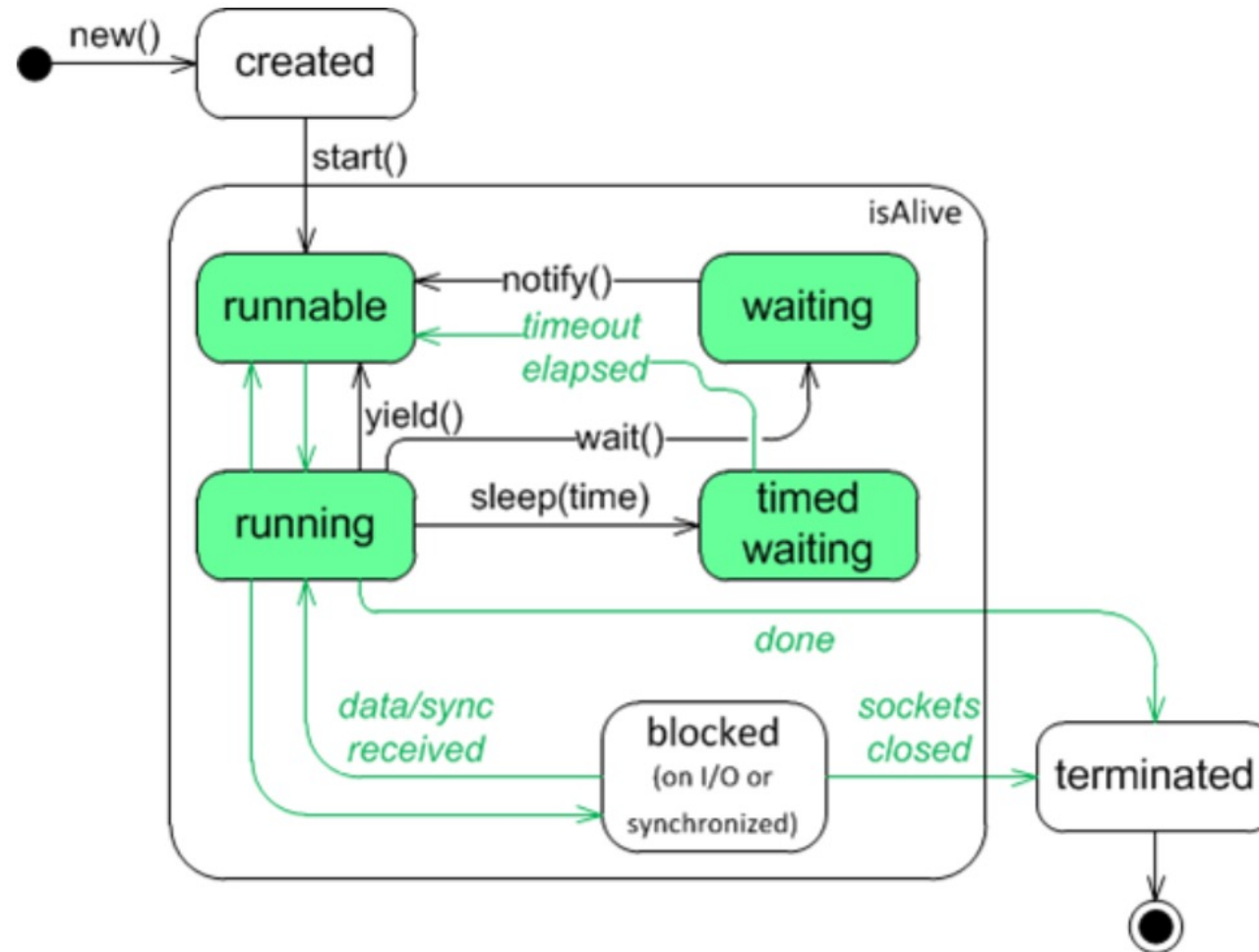
- Während der Thread `sleep` ausführt, kann eine `InterruptedException` ausgelöst werden, wodurch der interrupted status des Threads zurückgesetzt wird.

```
public class MachMal {  
  
    public static void pause(int zeit) {  
        try {  
            Thread.sleep(zeit);  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
    }  
}
```





Lebenszyklus eines Thread





Beispiel: Multithreading bei nur einem Core

- Bei 3 Threads A,B,C führt die CPU einige Zeiteinheiten z.B. Thread A aus.
 - Zustand der Berechnung von A speichern.
 - Zustand der Berechnung von B laden.
 - B weiter ausführen.
 - Zustand der Berechnung von B speichern
 - Zustand der Berechnung von C laden
 - C weiter ausführen
 - Zustand der Berechnung von C speichern
 - Zustand der Berechnung von A laden
 - A weiter ausführen
 - ...



Beispiel: Multithreading mit mehreren Cores

- Wie geht die CPU, die eine Reihe von Cores hat, mit Threads um?
 - Threads können auf die Cores verteilt werden und rechnen dort einige Zeiteinheiten.
 - Threads, für die aktuell kein freier Core vorhanden ist, werden in einer Warteschlange (Queue) gespeichert.
 - Ist ein Thread fertig (oder hat seine Zeiteinheiten aufgebraucht), wird ein neuer Thread aus der Queue ausgewählt.



Zugriff anderer Programmteile auf Threads

- Ein Thread-Objekt ist ein ganz normales Objekt einer Klasse, d.h. es kann Attribute und weitere Methoden haben.
- D.h. andere Programmteile können auf die Methoden des Threads zugreifen, wenn diese eine Referenz auf ein Thread-Objekt besitzen.
- Das gilt auch, nachdem die Ausführung des Threads, also `run()`, beendet ist.



Zugriff von Threads auf Objektdaten

- Threads können auf Objekte zugreifen.
- Wenn mehrere Threads Daten in einem Objekt halten, greifen alle auf **dasselbe** Objekt zu.
- Nutzen: **Kommunikation zwischen Threads**
 - Ein Thread legt Daten in einem Attribut `obj.message` ab; ein anderer Thread liest diese Daten.
- Achtung: Der Entwickler muss die Zugriffe **synchronisieren**.



Programmieren

11. Threads

11.3 Synchronisation von Threads



Synchronisation von Threads: Motivation

- Situation: Threads kommunizieren oder verwenden gemeinsame Daten
 - Wir benötigen Mechanismen, um die beteiligten Threads zu synchronisieren.
 - **Synchronisation** meint dabei, die **kausale Ordnung** zwischen Teilen von Threads herzustellen.
 - Beispiele:
 - **Kommunikation**: ein Thread wartet auf eine Nachricht von einem anderen Thread
 - **Gemeinsame Daten**: Verschiedene Threads wollen Transaktionen auf gemeinsamen Daten durchführen z.B. den Kontostand eines Kontos, auf dem verschiedene Threads Transaktionen (deposit, withdraw) durchführen.



Synchronisation von Threads: Beispiel

- Wenn Threads auf gemeinsame Daten (Objekte) zugreifen, müssen wir den Zugriff synchronisieren.
- Beispiel:
 - Klasse `AccountR` zur Verwaltung von Bankkonten.
 - Attribut `balance` enthält den aktuellen Kontostand.
 - Methode `deposit(int n)` erhöht den Kontostand um `n` Euro.
 - Methode `withdraw(int m)` bucht `m` Euro vom Konto ab.
 - `deposit` und `withdraw` von verschiedenen Threads für das gleiche `AccountR`-Objekt nebenläufig aufgerufen.



Klasse Account

```
public class AccountR {  
    private int balance;  
  
    AccountR(int i) { balance = i; }  
  
    public void deposit(int n) { balance = balance + n; }  
  
    public void withdraw(int m) { balance = balance - m; }  
  
    public int getBalance() { return balance; }  
}
```





Thread Customer zum Aufruf von Account



```
public class CustomerR extends Thread {  
    private boolean transaction; // true einzahlen, false abheben  
    private AccountR account;  
  
    CustomerR(boolean bool, AccountR acc) {  
        transaction = bool;  
        account = acc;  
    }  
  
    @Override  
    public void run() {  
        if (transaction) {  
            for (int i = 0; i < 1000; i++) { account.deposit(100); }  
        } else {  
            for (int i = 0; i < 1000; i++) { account.withdraw(100); }  
        }  
    }  
}
```



Starten der Threads

```
public class BankAccountR {  
    public static void main(String[] args) {  
        int count = 0;  
        System.out.println("Programm zum Erzeugen von Raceconditions");  
        for (int i = 0; i < 100_000; i++) {  
            AccountR acc = new AccountR(1000);  
            CustomerR trans1 = new CustomerR(true, acc);  
            CustomerR trans2 = new CustomerR(false, acc);  
            trans1.start(); trans2.start();  
            try {  
                trans1.join(); trans2.join();  
            } catch (InterruptedException e) { e.printStackTrace(); }  
            int res = acc.getBalance();  
            if (res != 1000) { count++; System.out.println(res); }  
        }  
        System.out.println("Anzahl Raceconditions: " + count);  
    }  
}
```



Beispielausgabe:

Programm zum Erzeugen ...

-5600
2500
66600
-1600
-47800
1100
2600
-1500
-12700
-10900
27600
-38000
-49700
-99000
-16500
-1300
1700
101000
-39700
-6100
48100
97800
101000
-10900

Anzahl Raceconditions: 24



Kritischer Bereich

- Der **kritische Bereich** ist das Codestück, in dem in beiden Threads mit `deposit` bzw. `withdraw` auf das Attribut `balance` zugegriffen wird.
- Dieser Zugriff wird nicht **atomar** ausgeführt.
- Die Zuweisung wird vom Compiler in mehrere Anweisungen zerlegt.
 - Lies `balance`
 - Addiere / subtrahiere 100
 - Speichere Ergebnis in `balance`



Race-Condition

- Nachdem Thread `trans1` `balance + 100` gerechnet hat und bevor die Zuweisung an `balance` erfolgt ist, wird er unterbrochen.
- Thread `trans2` liest `balance`, berechnet `balance - 100` und speichert das Ergebnis in `balance`.
- Nun wird `trans1` weiter ausgeführt und speichert sein Ergebnis in `balance`.
- Konsequenz:
 - die Transaktion von `trans2` geht verloren.
 - Das Ergebnis hängt von der Reihenfolge der Ausführung ab.
 - Das bezeichnet man als **Race-Condition**.
 - Solche Fehler passieren selten.



Race-Condition

- Im Beispiel werden 100.000 Accounts erzeugt und auf jedem dieser Accounts 1000-mal `deposit(100)` von `trans1` und 1000-mal `withdraw(100)` von `trans2` ausgeführt, um mit einer gewissen Wahrscheinlichkeit eine Race-Condition zu erreichen.
- Bei 100 Mio. Kontobewegungen wird meist nur eine zweistellige Zahl Races gezählt.
- Solche Fehler sind schwer reproduzierbar und damit auch schwer zu lokalisieren.



Synchronisation: Monitore in Java

- Da verschiedene Threads auf ein `AccountR`-Objekt zugreifen können, müssen wir die Zugriffe auf die Daten synchronisieren.
- Java bietet zur Synchronisation grundsätzlich drei Möglichkeiten:
 1. Schützen einer **kompletten Instanz-Methode**
 2. Schützen eines **Blocks** innerhalb einer Methode
 3. Schützen einer **statischen** Methode
- Um eine komplette Methode zu schützen, wird die Deklaration der Methode mit dem Schlüsselwort `synchronized` versehen
- Für alle Methoden einer Klasse (hier `AccountR`), die auf Attribute der Klasse zugreifen, wird i.a. geprüft, ob sie auf diese Weise geschützt werden müssen.

```
public synchronized void deposit(int n) { balance = balance + n; }  
public synchronized void withdraw(int m) { balance = balance - m; }  
public synchronized int getBalance() { return balance; }
```





Synchronisation: Monitore in Java

- Wenn eine mit `synchronized` geschützte Methode an einem Objekt aufgerufen wird, wird das Objekt, an dem die geschützte Methode aufgerufen wird, **gesperrt**, die Methode ausgeführt und anschließend die Sperre aufgehoben.
- Die Umsetzung erfolgt mit einem „Lock“ (Monitor), siehe `java.util.concurrent.locks` analog für einen geschützten Block
- Dadurch darf nur jeweils ein Thread zu einem Zeitpunkt eine mit `synchronized` geschützte Methode (bzw. Block) auf das `AccountR`-Objekt anwenden. Ein anderer Thread darf keine mit `synchronized` geschützte Methode an diesem (gesperrten) Objekt ausführen.
- Metapher: Schlüssel (engl. **Lock**) zu einem Raum, den man mitnimmt, wenn man hineingeht.
- Der Datenzugriff ist also gekapselt.
- Dieses Konstrukt zur Synchronisation des Zugriffs auf gemeinsam verwendete Daten nennt man **Monitor**.



Synchronisation eines Blocks

- Um kritische Codebereiche zu schützen, können auch **Anweisungsblöcke** mit dem Schlüsselwort `synchronized` eingeleitet werden:
- Die Sperre wird hier für ein Monitor-Objekt erworben, um die Anweisungen im Block ausführen zu können. Beim Austritt aus dem Block wird die Sperre für das Objekt aufgehoben.
- Der `synchronized`-Block bietet im Vergleich zur `synchronized`-Methode zwei Vorteile:
 - Möglichkeit zur Synchronisation eines „kurzen“ Codeabschnitts, exklusive Sperre wird für kürzere Zeit (als für die gesamte Methode) beansprucht
 - Als Sperrobjekte können neben `this` auch andere Objekte gewählt werden.

```
public int getBalance() {  
    synchronized (this) {  
        return balance;  
    }  
}
```





Synchronisation einer statischen Methode

- Synchronisation **statischer Methoden**:
 - Sperrobject bei synchronisierten Klassenmethoden (`static`-Methoden) ist das `Class`-Objekt der jeweiligen Klasse.
 - Zu jedem Zeitpunkt kann also nur immer eine synchronisierte Klassenmethode dieser Klasse ausgeführt werden.
 - Wenn eine Sperre auf Klassenebene erworben wurde und ein Objekt zu derselben Zeit eine synchronisierte Instanzmethode aufrufen will, wird die Instanzmethode blockiert, bis die Sperre auf Klassenebene aufgehoben wird.
- (wird eher selten verwendet)

```
public static void foo() {  
    synchronized (AccountR.class) {  
        // ...  
    }  
}
```



Synchronisation: wichtige Regeln

- Zwei Threads können dieselbe `synchronized`-Methode gleichzeitig ausführen, wenn sie für **verschiedene** Objekte aufgerufen wurde.
- Eine `synchronized`-Methode und eine Methode, die nicht mit `synchronized` gekennzeichnet ist, können zu derselben Zeit für **dasselbe** Objekt aufgerufen werden.
- Synchronisierte Methoden können andere synchronisierte Methoden **desselben** Objekts benutzen.
- Grundregel: Zugriffe synchronisieren
 - Wird von zwei oder mehreren Threads auf ein und dieselbe Variable zugegriffen, wobei mindestens ein Thread den Wert verändert, sollten **alle Zugriffe** auf diese Variable in `synchronized`-Methoden oder -Blöcken erfolgen.



Programmieren

11. Threads

11.4 wait und notify



Deadlock (Systemverklemmung)

- Was passiert, wenn ein Thread eine Methode des Monitors ausführt, die Ausführung aber nicht beenden kann, d.h. den Monitor blockiert?
- Beispiel:
 - In der Klasse `Account` kann mit `withdraw` nur Geld abgehoben werden, wenn der Kontostand dadurch nicht negativ wird. Es ist aber nicht immer genug Geld auf dem Konto.
 - Bei Aufruf der `withdraw`-Methode soll so lange gewartet werden, bis `balance >= m`.
 - Da `balance` nur durch `deposit` unter Nutzung des Monitors erhöht werden kann, ist das System verklemmt, wenn der wartende Thread den Monitor blockiert.
- Diese Situation bezeichnet man als **Deadlock (Systemverklemmung)**.



Wait and notify

- Diese Situation kann durch die Methoden `wait` und `notify` aus der Klasse `java.lang.Object` gelöst werden.
- `void wait() throws InterruptedException`
 - Der Thread legt sich schlafen und gibt den Monitor frei.
- `void notify()`
 - Weckt einen wartenden Prozess auf. Wartet kein Prozess, bleibt `notify()` wirkungslos.
- `void notifyAll()`
 - Weckt alle wartenden Prozesse auf.



Klasse Account mit wait / notify

```
public class Account {  
    private int balance;  
  
    Account(int i) { balance = i; }  
  
    public void deposit(int n) {  
        balance = balance + n;  
        notify();  
    }  
  
    ...  
}
```





Klasse Account mit wait / notify

```
public class Account {  
  
    ...  
    public synchronized void withdraw(int m) {  
        while (balance < m) {  
            try {  
                wait();  
            } catch (InterruptedException e) { /* ... */ }  
        }  
        balance = balance - m;  
    }  
}
```





Wait and notify

- `wait()` sollte immer innerhalb einer Schleife aufgerufen werden, in der man die Bedingung prüft, auf die der Thread wartet.
- Beim Aufwecken des Threads durch `notify()` ist nicht garantiert, dass die Bedingung erfüllt ist.
- In der Schleife prüft der Thread die Bedingung daher noch einmal und legt sich ggf. wieder mit `wait()` schlafen.
- Wenn ein Thread A aufgeweckt wird, betritt er den Monitor an der Wartestelle neu.
- Wenn noch ein anderer Thread B im Monitor ist, der A mit `notify()` geweckt hat, muss Thread A warten, bis B den Monitor verlässt.



Wait and notify

- `wait()` und `notify()` sollten nur von einem Thread aufgerufen werden, der den Monitor dieses Objektes hält; dies ist auf drei Weisen möglich:
 - Indem eine synchronisierte Instanzmethode dieses Objektes ausgeführt wird,
 - Indem ein synchronisierter Block ausgeführt auf diesem Objekt ausgeführt wird,
 - Für Objekte des Typs `Class`, indem eine synchronisierte statische Methode der Klasse ausgeführt wird.
 - Andernfalls wird eine `IllegalMonitorStateException` ausgelöst.
- Statt `notify()` kann man auch `notifyAll()` verwenden, wenn möglicherweise mehrere Threads warten.
- Der Aufruf von `wait()` muss in einen `try-catch`-Block geschachtelt werden.
- Falls der Thread beliebig lange auf das `notify()` wartet, kann man mit `Ctrl-C` eine `InterruptedException` auslösen, die behandelt werden muss.
- Das erfolglose Warten auf ein `notify()` bezeichnet man als **Starvation**. Der Prozess „verhungert“.



Programmieren

11. Threads

11.5 Java Concurrency Framework



Java Concurrency Framework

- Das Thema Nebenläufigkeit mit Threads ist sehr umfangreich
- Nur grundlegende Konzepte besprochen, aber viele Details und Erweiterungen vorhanden
 - Prioritäten von Threads
 - Scheduling mit Timer
 - Executoren
 - und vieles mehr
- Einige Konzepte gelten mittlerweile als veraltet oder nicht fein-granular genug, Java API bietet mit dem Java Concurrency Framework Alternativen im Paket `java.util.concurrent`



Java Concurrency Framework: Beispiele

- `java.util.concurrent.atomic`
 - enthält Datentypen für Thread-Programmierung mit atomarem Zugriff (Thread-sicher)
- `java.util.concurrent.locks`
 - bietet Frameworks für Locking und Warten, umfangreicher als `wait` und `notify`
- `java.util.concurrent.Semaphore`
 - ist eine Implementierung von Semaphoren (Eintritt erlaubt für bestimmte Anzahl Threads)
- `java.util.concurrent.BlockingQueue`
 - und weitere Datenstrukturen mit Thread-sicherem Zugriff.



Atomic-Variablen

- Auszug aus der API-Dokumentation
 - Package `java.util.concurrent.atomic`: A small toolkit of classes that support lock-free thread-safe programming on single variables.

```
import java.util.concurrent.atomic.AtomicInteger;

public class Account {
    private AtomicInteger balance = new AtomicInteger(0);

    Account(int i) {
        balance.set(i);
    }

    public void deposit(int n) {
        balance.getAndAdd(n);
    }

    ...
}
```





Locks

- Anstatt `synchronized` zu nutzen, kann alternativ auch ein Lock (Schloss) verwendet werden.
- Locks bieten mehr Funktionalität und feinere Steuerung als die impliziten Monitore.

```
public class MyLockCounter {  
    ReentrantLock lock = new ReentrantLock();  
    int count = 0;  
  
    void increment() {  
        lock.lock();  
        try {  
            count++;  
        } finally {  
            lock.unlock();  
        }  
    }  
}
```





Callables und Futures

- Ein `Runnable` kapselt einen Task, der asynchron läuft. Es wird die Methode `run()` implementiert, die keine Rückgabe vorsieht.
- Ein `Callable` ist einem `Runnable` ähnlich, hat aber einen Rückgabewert.
- Da die Methode auch asynchron läuft, ist das Ergebnis irgendwann in der Zukunft verfügbar.
- Das Ergebnis wird in ein `Future` gekapselt.
- Die Methode `get()` liefert das Ergebnis, oder blockiert, bis es berechnet ist.
- Die Methode `isDone()` prüft, ob die Berechnung fertig ist.



Fibonacci mit Callables and Futures

```
public class CallableFibonacci implements Callable<Integer> {  
    int n;  
  
    public CallableFibonacci(int n) { this.n = n; }  
  
    @Override  
    public Integer call() throws Exception {  
        return fibonacci(n);  
    }  
  
    private Integer fibonacci(int n) {  
        if (n <= 2) {  
            return 1;  
        } else {  
            return fibonacci(n - 1) + fibonacci(n - 2);  
        }  
    }  
}
```





Fibonacci mit Callables and Futures



```
public class FibCall {  
    public static void main(String[] args) throws Exception {  
        Callable<Integer> fibo = new CallableFibonacci(45);  
        FutureTask<Integer> task = new FutureTask<>(fibo);  
        Thread t = new Thread(task);  
        t.start();  
        while (true) {  
            System.out.println("In Schleife");  
            Thread.sleep(500);  
            if (task.isDone()) { break; }  
        }  
        System.out.println("Ergebnis: " + task.get());  
    }  
}
```



Fibonacci mit Callables and Futures



```
public class MoreFibCall {  
  
    public static void main(String[] args) {  
        ThreadPoolExecutor executor = (ThreadPoolExecutor) Executors.newFixedThreadPool(4);  
  
        List<Future<Integer>> resultList = new ArrayList<>();  
  
        Random random = new Random();  
        for (int i = 0; i < 4; i++) {  
            Integer number = random.nextInt(40,45);  
            CallableFibonacci fib = new CallableFibonacci(number);  
            Future<Integer> result = executor.submit(fib);  
            resultList.add(result);  
        }  
        for (Future<Integer> future : resultList) {  
            try {  
                System.out.println("Future result is - " + future.get() +  
                                   "; And Task done is " + future.isDone());  
            } catch (InterruptedException | ExecutionException e) { e.printStackTrace(); }  
        }  
        executor.shutdown();  
    }  
}
```



Übung

- Entwickeln Sie ein nebenläufiges Producer / Consumer Szenario.
- Es soll Produkte geben, die einen Namen bekommen.
- Producer erzeugen diese Produkte und Consumer konsumieren diese Produkte.
- Wenn ein Producer ein Produkt erzeugt hat, speichert er es in einer gemeinsam genutzten Datenstruktur MessageBuffer und pausiert danach für eine einstellbare Zeitspanne.
- Der Consumer konsumiert Produkte aus dieser gemeinsam genutzten Datenstruktur und pausiert ebenfalls eine einstellbare Zeitspanne.
- Consumer und Producer sollen Runnable implementieren und als Threads ablaufen.
- Die interne Speicherung der Produkte im MessageBuffer soll zunächst ohne weitere Maßnahmen als ArrayList erfolgen, wobei eine maximale Anzahl von Produkten angegeben werden kann.



Übung

- Lassen Sie Ihre Implementierung mit einem Producer und einem Consumer laufen, die jeweils 20 Produkte erzeugen bzw. konsumieren.
- Die maximale Kapazität des MessageBuffer soll 5 sein.
- Welche Beobachtung können Sie manchem wenn der Producer länger pausiert als der Consumer?
- Wie sieht es in dem Fall aus, dass der Consumer länger pausiert?
- Erstellen Sie eine weitere Implementierung des MessageBuffer mithilfe von Semaphoren und einem Lock, so dass die 20 Produkte erzeugt und konsumiert werden und die maximale Kapazität des MessageBuffer von 5 berücksichtigt wird. Lesen Sie zu Semaphoren die entsprechenden Teile der Java API. Die interne Datenstruktur zur Speicherung soll eine ArrayList bleiben.
- Erstellen Sie eine weitere Implementierung des MessageBuffer mithilfe einer gegen konkurrierenden Zugriff geschützten Java API Datenstruktur (siehe ebenfalls Java API).